Erdős problem 852: a published constant refuted at its twelfth digit and identified as a floating-point product, the certified correction, and the record data nobody had compared

Carlos Toledo*

October 2026 — draft v0.1 for review; every number interpolated from the record; machine-derived, not peer-reviewed; author list to be agreed

Abstract

Erdős problem 852 asks for the growth of $h(x)$, the longest run of pairwise-distinct consecutive prime gaps $d_n, \dots, d_{n+h-1}$ with $n < x$, and in particular whether $h(x) = o(\log x)$. Its discussion thread conjectures $h(x) \sim c_0 \log x$ and rests the conjecture on two constants produced with a language model and posted as bare decimals: $c_0 = 1.32322827686395\ldots$, the positive root of a rate function $I_0(c) = 1$, and $C_* = 0.0752403861777\ldots$, an Euler product over the odd primes. We replace both by certified interval enclosures. The root is enclosed to 61 decimal places, with uniqueness proved; the product is enclosed to width 3.2×10^{-16} over 1,857,858 odd primes with the infinite tail bounded by an elementary integral. The posted c_0 is a correct rounding to 14 places, though not a truncation. The posted C_* is refuted from its 12th decimal place by a single exact integer inequality over the odd primes to 400,000, re-proved by a stdlib verifier and by the Lean 4 kernel. The mechanism is identified and measured: a naive IEEE-754 double product reproduces the posted decimal to every place, because the factor $1 + (p-1)^{-3}$ rounds to exactly 1.0 for $p - 1 \geq 208{,}064$ and 87.5% of the factors vanish, so that no loop bound changes the result; a forward `log1p` sum absorbs its small terms the same way and also lands outside the enclosure. The exact record data of the problem, in the OEIS since 2002 (A053597, A078515, A079007, A079889), had never been compared with the conjecture. An exhaustive integer scan to 5×10^{11} reproduces every published term and extends the record to a run of 31 gaps, re-proved independently. Under the reading the statement gives, x bounding the index n, the ratio $h(x)/\log x$ lies between 1.22 and 1.41 over 8 decades and the certified c_0 lies inside 12 of 22 plateau bands; under the alternative reading, x bounding the prime, it lies between 0.98 and 1.14. Nothing asymptotic is claimed. The contribution is the decision grammar — a verdict with its witness, and refusal as a result — and a public record that re-decides itself at every build.

*Independent researcher. Contact: carlos@carlostoledo.co. This work was produced with substantial AI assistance in a certified-verification setting: constructions and exposition were proposed with the help of large language models, and every quantitative claim is decided by outward-rounded interval arithmetic or exact rational arithmetic, with adversarial controls that must fire for the certificate to be accepted. The certificates, not the models, are the evidence, and they are public and re-runnable. Machine-derived and not peer-reviewed.

1 Introduction

Let p_n be the nth prime and $d_n = p_{n+1} - p_n$. As published on the problem page [1], Erdős problem 852 reads: let $h(x)$ be maximal such that for some $n < x$ the numbers $d_n, d_{n+1}, \dots, d_{n+h(x)-1}$ are all distinct; estimate $h(x)$; in particular, is $h(x) > (\log x)^c$ for some constant $c > 0$, and is $h(x) = o(\log x)$? The page keys the source as Erdős's 1985 paper [Er85c] [2], records that Brun's sieve gives $h(x) \to \infty$, and states that the problem cannot be resolved with a finite computation. Nothing in this paper contradicts that last statement.

The thread. The problem's discussion thread [3] holds, as of the snapshot pinned on 25 August 2026, 7 comments. Three of them carry quantitative claims obtained with language models, which we quote as recorded there and do not re-derive: a lower bound $h(x) \gg (\log x)^{1/3}$ by a Selberg-sieve count of blocks with a repeated gap; an unconditional upper bound $h(x) \ll x^{0.41+\epsilon}$ from a second-moment estimate for prime gaps, and a conditional $h(x) \ll x^{0.26+o(1)}$; and, on 24 April 2026, a conjectured asymptotic. In the iid geometric-gap model the probability that $H = cL$ consecutive gaps are distinct decays like $\exp(-L\, I_0(c))$ with

$$I_0(c) = c + c \log \frac{e^{2c} - 1}{2c} + \tfrac{1}{2} \operatorname{Li}_2\big(1 - e^{2c}\big)\,, \tag{1}$$

so the longest run among x candidate starts should be $c_0 \log x$ with $I_0(c_0) = 1$; the thread gives $c_0 = 1.32322827686395\ldots$ and the conjecture $h(x) \sim 1.32323 \log x$. Passing from the model to the primes adds a singular-series term $J(c) = C_* c^2 + O(c^3)$ with

$$C_* = \tfrac{1}{2}\Big(\prod_{p \geq 3} \Big(1 + \frac{1}{(p-1)^3}\Big) - 1 \Big) = 0.0752403861777\ldots, \tag{2}$$

and the thread's conditional argument against $h(x) = o(\log x)$ rests on $(\frac{1}{2} + C_*)c^2 < 1$ for small c. Both decimals were posted without an error bound and without the computation that produced them.

The data. The exact record data of this problem have been in the On-Line Encyclopedia of Integer Sequences since 2002 [4]: A053597 is the run length starting at each index, A078515 its record indices, A079889 the primes at those indices, and A079007 the smallest prime opening a run of each length; the last extension of the record sequences dates from 2012. The problem page lists A001223, A053597 and A078515 in its own OEIS box, and a thread comment of December 2025 remarks that A078515 "looks exponential". The constant and the data had not been put side by side.

What this paper does. It replaces the two decimals by certified enclosures (Theorems 1 and 2), decides the two posted decimals against them (Propositions 1 and 2), identifies and measures the computation that produced the wrong one (Proposition 3), recomputes the record data exactly to 5×10^{11} and extends them (Theorem 3 and Proposition 4), and reports what the data say about the conjecture under the two readings of the statement (Observation 1). It closes with the failure taxonomy the episode belongs to, written for people who build evaluations on mathematical ground truth.

Throughout, a quantity is *certified* when an interval $[\ell, u]$ with $\ell \leq \text{truth} \leq u$ has been computed with every rounding directed outward and every truncated series carrying an explicit

remainder bound, or when a decision has been made in exact integer or rational arithmetic; a certified enclosure is a theorem about the real number it brackets. Interval arithmetic is not the contribution of this paper. The contribution is the grammar in which the results are stated — a verdict (verified, verified as a rounding, refuted, undecided) with its witness, and refusal as a result in its own right — together with a public record that re-derives every number quoted here at every build and refuses to build when a record no longer supports a sentence [13]. Results of other people are re-decided, not corrected by hand.

2 Statement of results

Theorem 1 (the rate constant)**.** *The equation $I_0(c) = 1$, with I_0 as in* (1)*, has exactly one root $c_0 > 0$. Let*

$$\ell = 1.32322827686394946902896939329746346135865351260047599684898 56, \qquad u = \ell + 10^{-40}.$$

Then $I_0(\ell) < 1 < I_0(u)$, so $\ell < c_0 < u$, and the first 61 decimals of c_0 are

$$c_0 = 1.32322827686394946902896939329746346135865351260047599684898 56\ldots$$

Theorem 2 (the singular-series constant)**.** *With C_* as in* (2)*,*

$$0.0752403861783092455893 \;\le\; C_* \;\le\; 0.0752403861783095651674,$$

an enclosure of width 3.20×10^{-16}; in particular $C_* = 0.075240386178309\ldots$

Proposition 1 (the posted C_* is refuted)**.** *Let $N/D = \prod_{3\le p\le 400{,}000} \left(1+(p-1)^{-3}\right)$ in lowest or any terms, a product over 33,859 odd primes. Then $5\,(N-D)\cdot 10^{12} > 752403861778\,D$, that is, $\frac{1}{2}(N/D-1) > 0.0752403861778$. Since every omitted factor exceeds 1, $C_* > 0.0752403861781751$, which lies above the posted decimal 0.0752403861777 whether that decimal is read as a truncation (truth in $[0.0752403861777, 0.0752403861778]$) or as a rounding (truth within 5×10^{-14} of it). The posted value is wrong from its 12th decimal place, the 11th significant digit; 11 of its 13 decimal places are correct.*

Proposition 2 (the posted c_0 is a rounding, not a truncation)**.** *The decimal 1.323228276863 95 is the correct rounding of c_0 to 14 places, but its digits are not the leading digits of c_0: the expansion continues $1.323228276863949469\ldots$, so a trailing ellipsis after 1.323228276863 95 is wrong by half a unit in the last place.*

Proposition 3 (the mechanism, measured)**.** *In IEEE-754 binary64 arithmetic with round-to-nearest [6], $1 + (p-1)^{-3}$ evaluates to exactly 1.0 if and only if $(p-1)^3 \ge 2^{53}$, that is $p - 1 \ge 208{,}064$ (here 2^{53} is 9,007,199,254,740,992); the first such prime is 208,067. Evaluating $\frac{1}{2}(\prod -1)$ with double-precision factors over the odd primes to 2,000,000 gives 0.0752403861777419, which rounds to the posted decimal 0.0752403861777 at 13 places; 130,293 of its 148,932 factors (87.5%) are exactly 1.0, and the same sixteen-digit value is obtained with the loop bound at 4×10^5 and at 1×10^7. The discarded mass $\sum_{p\ge 208{,}067}(p-1)^{-3}$ of the logarithm of the product, 9.07×10^{-13}, predicts a deficit of 5.22×10^{-13} in C_*; the measured deficit of the naive value against the enclosure is 5.68×10^{-13}.*

Theorem 3 (a run of 31 distinct gaps)**.** *The prime* $p = 263{,}552{,}821{,}783$ *is followed by 31 consecutive prime gaps that are pairwise distinct, namely, in order, 76, 8, 34, 36, 26, 106, 38, 82, 18, 54, 50, 16, 62, 70, 14, 48, 42, 64, 6, 104, 24, 12, 66, 22, 30, 96, 2, 10, 74, 4, 32, spanning the primes 263,552,821,783 to 263,552,823,109; the next gap is 4, which already occurs, so the run has length exactly 31. Consequently* $h(x) \geq 31$ *for every* $x > 10{,}435{,}962{,}861$*, the index of* p*.*

Proposition 4 (decided by the exhaustive scan)**.** *An integer scan of all primes below* 5×10^{11} *finds 28 record indices (Table 3); the first 27 agree term for term with A078515 and A079889, the opening primes by run length agree with A079007 at every published term, and A053597 is reproduced at its 105 published terms. The index 10,435,962,861 of Theorem 3 is the smallest opening a run of length 31, and no run of 32 or more pairwise-distinct consecutive gaps lies among the primes below* 5×10^{11}*. These two minimality statements rest on one implementation and are not independently re-proved; the agreement with the published terms is their evidence.*

Observation 1. Over $x = 10^4, \ldots, 10^{11}$, with x bounding the index n as the statement says, $h(x)/\log x$ lies in $[1.22, 1.41]$ with mean 1.307, and the certified $c_0 = 1.3232$ lies inside 12 of the 22 closed plateau bands of $h/\log x$, 7 bands lying wholly below it and 3 wholly above. With x bounding the prime p_n instead, the ratio lies in $[0.98, 1.14]$ with mean 1.090, and is 1.1450 at 10^{11}. This is a finite-range statement; see Section 4.4 for what it does and does not say.

3 The instrument

Directed-rounding big floats. A number is $m \cdot 2^e$ with m an arbitrary-precision integer; an interval is a pair of such numbers; every operation rounds the lower end toward $-\infty$ and the upper end toward $+\infty$ at a chosen precision P, so containment is preserved by construction. Integers and dyadic constants pass through exactly. The transcendental functions used (exp, log, π by Machin's formula, Li_2 on the small disk) are truncated series with an explicit remainder added to the interval. Doubles stop near fourteen digits and exact rationals cannot carry a product of 1,857,858 factors (the numerator over the primes to 400,000 alone has 520,354 digits); the directed dyadic layer is the missing middle, and it is itself checked against exact rationals on random operands, with the rounding direction mutation-tested, before any constant is decided [13].

The root (Theorem 1). On the bracket the dilogarithm argument $1 - e^{2c} \approx -13$ lies far outside the unit disk, so (1) is evaluated through the inversion identity $\mathrm{Li}_2(-x) + \mathrm{Li}_2(-1/x) = -\pi^2/6 - \frac{1}{2}\log^2 x$ for $x > 0$ [5, eq. (1.12)], with $x = e^{2c} - 1$, which leaves Li_2 at $-1/x \approx -0.076$ where the defining series has a geometric tail bound:

$$I_0(c) = c + c\,(\log x - \log 2c) - \tfrac{\pi^2}{12} - \tfrac{1}{4}\log^2 x - \tfrac{1}{2}\,\mathrm{Li}_2(-1/x).$$

The root is bisected from $[1.25, 1.375]$ for 200 steps at 320 bits, each step a certified strict inequality $I_0(m) < 1$ or $I_0(m) > 1$, which leaves a bracket of width below 7.8×10^{-62}. Uniqueness is elementary: differentiating (1) with $\mathrm{Li}_2{}'(z) = -\log(1-z)/z$ and $1 - z = e^{2c}$ cancels every term but

$$I_0'(c) = \log \frac{e^{2c} - 1}{2c} > 0 \quad (c > 0), \qquad \text{since } e^{2c} > 1 + 2c,$$

and $I_0(c) \to 0$ as $c \to 0^+$. The strict inequality $e^{2c} > 1 + 2c$ is certified on the bracket as well, and the derivative identity — the one consumed calculus fact — is checked numerically in the

battery against a centred difference. The detached certificate exports the coarser 40-decimal window $[\ell, u]$ of Theorem 1, and a Python standard-library verifier that shares no code with the instrument re-evaluates I_0 at both ends at 130 digits (correctly rounded exp and ln, π by Machin with the alternating-series remainder, Li_2 by its series with a geometric remainder) and finds $I_0(\ell) - 1 < 0$ and $I_0(u) - 1 > 0$ with margins 8.6×10^{-41} and 7.4×10^{-41}, refusing any margin below 10^{-50} rather than deciding inside its own noise.

The product (Theorem 2). The partial product over the odd primes $p \le L = 30{,}000{,}000$ is accumulated in the directed layer at 192 bits, one exact rational factor $(q^3+1)/q^3$, $q = p-1$, per prime. The tail is a certified multiplier: since $p \ge L+1$ gives $p - 1 \ge L$,

$$\sum_{p>L} \frac{1}{(p-1)^3} \le \sum_{m \ge L} \frac{1}{m^3} \le \int_{L-1}^{\infty} \frac{dt}{t^3} = \frac{1}{2(L-1)^2} =: S,$$

$$1 \le \prod_{p>L} \left(1 + \frac{1}{(p-1)^3}\right) \le e^S \le 1 + S + S^2$$

(the last step for $S \le \frac{1}{2}$); at this L, S is 5.56×10^{-16}. The same engine is calibrated at every run against a product with a known value, $\prod_{p \ge 3} \left(1 + (p^2-1)^{-1}\right) = \frac{3}{4}\zeta(2) = \pi^2/8$ (Euler's product [7]), whose enclosure must contain $\pi^2/8$; a mutation control zeroes the tail bound and must watch that containment fail. A tail that cannot be missed is not being checked.

Deciding a posted decimal. A decimal d with k places posted with a trailing ellipsis is a claim under one of two conventions: truncation, truth in $[d, d + 10^{-k}]$, or rounding, truth in $[d - \frac{1}{2}10^{-k}, d + \frac{1}{2}10^{-k}]$. Against an enclosure $[\ell, u]$, decided by exact rational comparison, the verdict is VERIFIED if $[\ell, u]$ lies inside the truncation window, VERIFIED AS A ROUNDING if it lies inside the rounding window but not the truncation window, REFUTED if it is disjoint from the union of both windows, and UNDECIDED otherwise. The grammar matters: Proposition 2 is a verdict a digit comparison cannot express, and UNDECIDED is a result, not a failure.

The refutation in exact integers (Proposition 1). The posted C_* is not refuted by trusting the enclosure. The exact partial product N/D over the odd primes to 400,000 is a strict lower bound of the infinite product, and one integer inequality, $5\,(N - D) \cdot 10^{12} > 752403861778\,D$, places $\frac{1}{2}(N/D - 1)$ above the upper edge of both windows. The stdlib verifier re-derives N and D by a balanced product tree and decides the inequality in 1.4 s; it also derives an exact upper window from S and checks that the instrument's enclosure lies inside it. Its 17 checks include 4 controls that must fire (a forged π must break the root window; the true 13-decimal truncation of C_* must not be refuted; a claim above the exact upper window must be refuted through the upper path; a forged source hash must be rejected), and it prints the SHA-256 of the certificate file it checked (`13cfc7fa212a…`).

The kernel bridge. The same inequality is stated and proved in Lean 4 [11] with Mathlib [12] (toolchain `leanprover/lean4:v4.33.1`): the odd primes to 400,000 are data in 240 chunk modules; a ten-line trial-division primality test carries a machine-checked correctness theorem against `Nat.Prime`; the kernel re-proves primality, strict ascent and oddness of every listed prime, two rewriting theorems that the tree of chunk subproducts equals the product over the list, and

the inequality `cstar_refuted` by `decide`. The prime list needs no trust: an omitted prime only weakens a lower bound, and a planted composite, a broken ascent or an inflated claim each fail the build, which three forged variants demonstrate at every check. Outside the kernel remain the one-paragraph monotonicity argument for a sub-product of factors exceeding one and the transcription of the posted decimal.

The record scan (Theorem 3, Proposition 4). A segmented sieve streams the primes below 5×10^{11} into a two-pointer window: for each gap index j, the left end is the smallest start such that the gaps in the window are pairwise distinct, advanced past the previous occurrence whenever a gap value repeats; a record is the first index whose window exceeds every earlier length. Only a last-seen table of 1,024 gap values is held, so memory is independent of the limit; the run to 5×10^{11} took 4,042 s. No floating-point number enters it. Two gates precede any use of the output: a structural validation (indices, lengths and opening primes increasing, length steps at most three, a plausibility ceiling) that needs no reference, and the term-for-term comparison with the pinned OEIS bytes. The validation exists because a first deep run stored gap indices in 32-bit integers, wrapped at the 2^{31}st prime and produced a "record" of more than ten million gaps beyond the last published term, where a prefix comparison would have passed it; a reference check covers only the head of a deep run. Every record past the published terms is re-proved by a program sharing no line with the scan — a deterministic Miller–Rabin test on the twelve prime bases up to 37, which is deterministic far beyond this range [10], next-prime by stepping, and a set for distinctness — which certifies the existence claim and is silent on minimality; the last published term is re-proved the same way as a control.

4 Results

4.1 The two constants

Table 1 states the two verdicts. The posted c_0 survives as a rounding (Proposition 2): the decision requires knowing the 14th decimal and the ones after it exactly, which the 61-place enclosure does. The posted C_* is refuted (Proposition 1); the exact partial product to 400,000 already gives $C_* > 0.0752403861781751$, with no tail bound and no rounding, and the certified enclosure lies 6.09×10^{-13} above the posted decimal.

constant	posted	certified	method	verdict
c_0	$1.32322827686395\ldots$	$1.32322827686394946902\ldots$	root of $I_0 = 1$, 200 certified bisections	verified as a rounding
C_*	$0.0752403861777\ldots$	$0.075240386178309\ldots$	product over 1,857,858 odd primes, tail proved	refuted at decimal 12

Table 1: The two constants of the thread's conjecture, decided against certified enclosures. Both posted decimals are transcribed from the pinned bytes of the thread, which are re-hashed at every build.

4.2 The mechanism

The wrong digits are not near the truth by accident. Table 2 lists four computations over the same odd primes to 2,000,000. The naive double product is the posted decimal to every place,

and it does not move with the loop bound (Proposition 3): the 18,639 primes below the threshold are the whole computation, and any loop that runs further multiplies by 1.0. Two independent float implementations of this product agree with each other and not with the truth, and a longer loop reads as convergence. The discarded mass of the logarithm accounts for the deficit to within 8%; the remainder is the ordinary rounding of the kept factors.

The route a careful floating-point programmer would take, summing $\log(1 + a_p)$ with `log1p` so that no factor is lost to the addition of 1, does not repair it. A forward sum of the 148,932 terms absorbs every term smaller than half an ulp of the running sum (about 1.4×10^{-17} for a sum near 0.14): from $p = 416{,}147$ on, 113,846 terms (76.4%) add nothing, and the result lands 9.4×10^{-14} below the certified enclosure — closer than the product, and still outside. The same terms added with Kahan's compensated summation [8, 9] land inside. The lesson is not that doubles are inadequate; it is that a float result carries no statement of its own error, and only a method that does — directed rounding, or exact arithmetic — can decide a thirteenth decimal.

computation over the odd primes to 2,000,000	value	against the enclosure
naive double product (87.5% of the factors equal to 1.0)	0.0752403861777419	the published decimal to all its places; outside
forward double `log1p` sum (76.4% of the terms absorbed)	0.0752403861782102	outside, by 9.4×10^{-14}
compensated `log1p` sum (Kahan)	0.0752403861783045	inside; a float, so uncertified
certified enclosure, same loop bound	[0.0752403861783044, 0.0752403861783764]	the authority

Table 2: The mechanism, re-measured at every build. The certified enclosure at this loop bound already refutes the posted decimal and verifies the 13-decimal truncation 0.0752403861783 of the true value.

4.3 The record data

Table 3 lists every record index of the scan to 5×10^{11} together with the plateau it opens: 28 distinct indices (31 records, since an index can set two lengths at once, as do 19,205 (lengths 14 and 15) and 1,311,699,253 (lengths 28 and 29), which is why A079007 repeats a prime at each). The first 27 agree with A078515 and A079889 term for term, the opening primes by length agree with A079007 at all 31 published terms $n = 0, \ldots, 30$, and the 105 published terms of A053597 are reproduced from the primes directly. Two independent witnesses, a sequence computed by other people between 2002 and 2012 and a scan written from the statement, agree on the same integers; that agreement is the one hard result of this section that does not depend on either.

The scan then passes the last published term. A079007 ends at a run of 30 gaps opening at 196,948,778,371 (index 7,889,803,997); the next record is Theorem 3, a run of 31 opening at 263,552,821,783 (index 10,435,962,861), the largest gap in it being 106. Anyone can check the theorem in a minute: the 31 numbers listed are the differences of the 31+1 consecutive primes from 263,552,821,783 to 263,552,823,109, they are pairwise distinct, and the next difference, 4, is among them. That the index is the smallest with this property is Proposition 4's claim, and the paper keeps the two apart: existence is re-proved by the independent verifier, minimality rests on the scan.

h	record index n	opening prime p_n	in OEIS	plateau end	$h/\log x$ at start	at end	$c_0 = 1.3232$
2	1	2	yes	—	—	—	—
3	7	17	yes	—	—	—	—
4	23	83	yes	—	—	—	—
5	30	113	yes	—	—	—	—
6	94	491	yes	—	—	—	—
7	219	1,367	yes	285	1.2924	1.2384	below c_0
8	279	1,801	yes	780	1.4144	1.2013	inside
9	773	5,869	yes	1,864	1.3512	1.1951	inside
10	1,856	15,919	yes	3,733	1.3278	1.2158	inside
11	3,724	34,883	yes	7,009	1.3373	1.2422	inside
12	6,999	70,639	yes	7,011	1.3552	1.3551	above c_0
13	7,000	70,657	yes	19,218	1.4680	1.3180	inside
15	19,205	214,867	yes	184,177	1.5207	1.2373	inside
16	184,163	2,515,871	yes	280,118	1.3197	1.2756	below c_0
17	280,103	3,952,733	yes	849,892	1.3553	1.2452	inside
18	849,876	13,010,143	yes	1,870,739	1.3184	1.2464	below c_0
19	1,870,722	30,220,163	yes	3,570,779	1.3156	1.2593	below c_0
20	3,570,761	60,155,567	yes	4,114,360	1.3255	1.3132	inside
21	4,114,341	69,931,991	yes	11,271,092	1.3789	1.2933	inside
22	11,271,072	203,674,907	yes	55,282,795	1.3549	1.2340	inside
23	55,282,774	1,092,101,119	yes	68,256,062	1.2901	1.2750	below c_0
24	68,256,040	1,363,592,621	yes	68,256,064	1.3305	1.3305	above c_0
25	68,256,041	1,363,592,677	yes	104,011,383	1.3859	1.3543	above c_0
26	104,011,359	2,124,140,323	yes	1,009,322,516	1.4084	1.2541	inside
27	1,009,322,491	23,024,158,649	yes	1,311,699,280	1.3023	1.2860	below c_0
29	1,311,699,253	30,282,104,173	yes	7,889,804,025	1.3813	1.2726	inside
30	7,889,803,997	196,948,778,371	yes	10,435,962,890	1.3164	1.3005	below c_0
31	10,435,962,861	263,552,821,783	**new**	open	1.3438	—	open

Table 3: Every record of the exhaustive scan to 5×10^{11}: the smallest index n at which h consecutive prime gaps are pairwise distinct, the prime there, whether the pair is in A078515/A079889 (the first 27 are; the opening primes by length are A079007), and the plateau of h it opens under the index reading, from index $n + h - 1$ to the given end, with the band of $h/\log x$ it occupies and where c_0 falls. Bands are reported from $h = 7$, below which $\log x$ is a poor guide; the last plateau is open at the edge of the scan. The lengths 14 and 28 are absent because the indices 19,205 (lengths 14 and 15) and 1,311,699,253 (lengths 28 and 29) set two records at once. The last row is Theorem 3.

4.4 The conjecture against the data, and the two readings

The statement bounds the *index*: "for some $n < x$". It is easy to read x as a bound on the size of the primes instead, and at finite range the two readings give different numbers. Table 4 and Figure 1(a) give $h(x)$ and $h(x)/\log x$ at the decades 10^4 to 10^{11} under both: under the index reading, $h(x)$ is the longest run that has closed by index x, and under the prime reading the longest run opening at a prime at most x (the opening prime is used, which can only favour this reading). Under the index reading the ratio stays in $[1.22, 1.41]$ and the predicted $c_0 \log x$ is within one unit of $h(x)$ at six of the 8 decades; under the prime reading it stays in $[0.98, 1.14]$ and shows no approach to c_0 over the range reached.

Sampling at decades has a bias the figure removes. h is a step function whose records are its jumps, so quoting $h/\log x$ at the records reads only the tops of the steps. Between one record closing and the next, h is constant while $\log x$ grows, and the ratio falls from a high at the start of the plateau to a low at its end; each closed plateau is therefore reported as the band

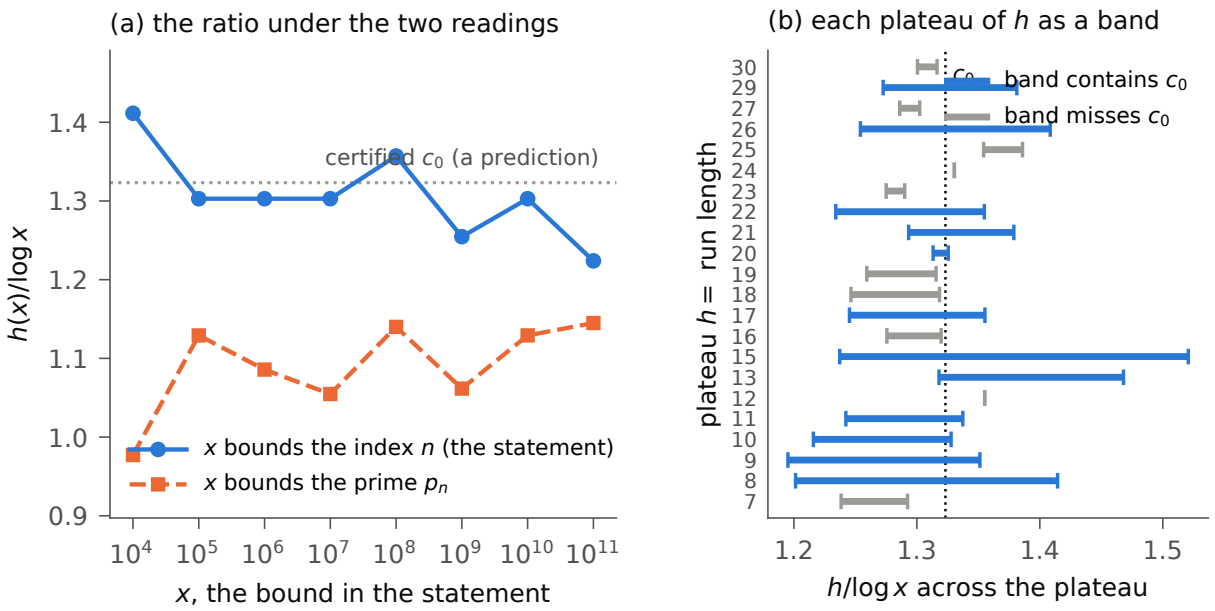


Figure 1: (a) $h(x)/\log x$ by decade under the two readings of the statement, with the certified c_0 as a dotted reference: a prediction, drawn differently from the measurements. (b) Each closed plateau of h as the band of $h/\log x$ it occupies, from its start (right end) to its end (left end); the dotted line is c_0. Drawn from `certs/erdos852-h-records.json` through the module the report page uses.

it occupies (Table 3, Figure 1(b)), and the honest question is whether c_0 lies inside the band. It does in 12 of 22 closed plateaus from $h = 7$ on; of the misses, 7 bands lie wholly below c_0 and 3 wholly above, 2 of the latter being plateaus of only 2 indices. The misses are shown, not smoothed.

What the comparison says must be stated with the same care as the verdicts. First, the two readings agree asymptotically: $\log p_n = \log n + \log\log n + O(1)$, so a ratio that tends to c_0 under one reading tends to c_0 under the other, and the data cannot distinguish the two limits. What they distinguish is finite-range behaviour: at the last record the factor $\log n/\log p_n$ is 0.877, and it tends to 1 only like $1 - \log\log p_n/\log p_n$, which is 0.124 there and decays as slowly as anything in analytic number theory. Second, the agreement under the index reading is consistency, not confirmation: eight decades and twenty-two plateaus are evidence about a conjecture that the problem page rightly says no finite computation can settle, and a second-order term of either sign could move the finite-range ratio by the amounts seen. What can be said is this: under the reading the statement gives, the ratio is already flat at the certified constant across everything computed, and under the other reading it is not; and Erdős's own question, whether $h(x) = o(\log x)$, is answered by these data in the direction of "no" as far as they reach, with the ratio neither decaying nor drifting. The thread's conjecture is recorded in the repository's formal statement of the problem in Lean (which formalises the index reading and states the boundedness of the run-length set as the named side-condition it is, since the supremum of an unbounded set is junk in Mathlib) precisely because it is incompatible with a "yes".

x	$h(x)$, index reading	$h/\log x$	$h(x)$, prime reading	$h/\log x$	$c_0 \log x$
10^4	13	1.4115	9	0.9772	12.19
10^5	15	1.3029	13	1.1292	15.23
10^6	18	1.3029	15	1.0857	18.28
10^7	21	1.3029	17	1.0547	21.33
10^8	25	1.3572	21	1.1400	24.37
10^9	26	1.2546	22	1.0616	27.42
10^{10}	30	1.3029	26	1.1292	30.47
10^{11}	31	1.2239	29	1.1450	33.52

Table 4: The same exact record data under the two readings of "for some $n < x$". The last column is the thread's prediction, $c_0 \log x$ with the certified c_0.

5 A taxonomy of failure, for people who grade mathematics

The C_* episode is the type specimen of a failure class, and the repository's records now hold three distinct classes, each decided, each with the detector that decides it. They are worth naming because the usual defences — rerun it, cross-check the digits, add compute — decide none of them.

1. *The artifact posted as truth.* The computation is silently wrong and its output is stable: every naive rerun reproduces the same digits, two independent float implementations agree with each other, and raising the loop bound changes nothing (Proposition 3). The posted value is the bug, printed, with 13 decimals of confidence when 11 were real. What decides it is an exact arbiter the value provably lies outside.

2. *The true computation under a false printed claim.* The numerics were right; the posted identity is not what they computed. The repository's specimen is a row of a Ramanujan Machine results sheet whose own polynomials converge to $2/(2\zeta(5) - 2\zeta(3) + 1)$ while the printed formula carries -1, a sign slip, with the printed convergent contradicting the row's own recurrence [14]. There is no second number to check digits against; what decides it is deciding the printed identity against a certified enclosure of the object the row defines, and then certifying the corrected identity on the same enclosure, so that the mechanism is proved rather than guessed.

3. *The coincidence that survives every screen.* Nothing was computed wrongly; the world contains near-misses deeper than any float. The repository's specimen is a published OEIS constant that agrees with $1/5$ to more significant digits than any decimal screen ever used to announce a discovery has asked for, and is not $1/5$ [15]. Probability arguments certify confidence, not truth; what decides it is an exact refutation at the full published precision, the only test with a zero false-accept rate.

What the three share: the failure is invisible from inside floating point, stable under repetition, and dressed as precision. For anyone building evaluations or verifiable-reward environments on mathematical ground truth, three consequences follow. Digit-matching ground truth inherits the failure class of whatever computed it: if a reference value came from a float pipeline, class 1 lives inside the answer key, and a model that reproduces the artifact grades as correct — the reference must carry its own certificate, not just its digits. Human graders cannot decide this

class either; no reviewer evaluates a product of 1,857,858 factors by eye, and the C_* decimal sat on a public thread from 24 April 2026 until the correction of 26 August 2026 [3]. And a verifier is trustworthy only if it can be shown failing: every instrument here ships controls that must fire (4 in the battery of 22 checks, 4 in the stdlib verifier, three forged variants rejected by the Lean kernel) and calibrations against known answers that must pass before any novel claim is decided. An answer key that has never rejected a plausible forgery has an unknown false-accept rate.

6 What is not claimed

Nothing asymptotic is proved. The problem page states that 852 cannot be resolved by a finite computation, and this paper agrees: Observation 1 is consistency over a finite range, and Section 4.4 says why a finite range cannot separate the two readings' limits. The minimality statements of Proposition 4 rest on one exhaustive scan and are not independently re-proved; the existence statement of Theorem 3 is. The correction of C_* concerns the numerical value only: it does not touch the structure of the thread's conditional argument, and since $\frac{1}{2} + C_*$ changes in its 12th decimal the qualitative conclusion $I_0(c) + J(c) < 1$ for small $c > 0$ is unaffected; the thread's sieve bounds are quoted as recorded and were not re-derived here. The mechanism identifies a computation, not an author: it says what arithmetic produced the posted digits, not who ran it or what tool proposed it, and the posted c_0 is correct to the places it was given. The repository's stack and this paper share an author, which is why the refutation is also carried by a stdlib verifier and by a kernel that is not ours; the enclosures of Theorems 1 and 2 beyond the refutation inequality rest on the directed-rounding layer and its battery, and an outside re-derivation of them at the stated widths is invited.

Reproduction

Every constant in this paper is interpolated from a certificate record in the `cert-machine` repository, so the document cannot quote a number the machine did not certify. The detached certificate, `certs/erdos852-certificate.json`, was exported by `tools/export-erdos852-certificate.js` at commit `be82e12`; the command

```
python3 tools/verify_erdos852.py certs/erdos852-certificate.json --sources corpus/sources
```

re-decides both constants in 1.4 s with the standard library only. The instrument is `instruments/erdos852/` over `instruments/bigfloat/`, with `families/erdos852-constants.js` as the four objects; its battery (`instruments/erdos852/battery.js`, run with `node`) has 22 checks. The kernel bridge is `lean/erdos852/`; `bash lean/erdos852/check.sh` builds it and must see the three forgeries rejected. The record scan is `instruments/erdos852h/h.js` (`node instruments/erdos852h/h.js 5e11` reproduces `certs/erdos852-h-records.json` in about 67 minutes), its analysis `analyse.js`, and

```
node instruments/erdos852h/verify-record.js 263552821783 31
```

re-proves Theorem 3. Sources are pinned by SHA-256 in `corpus/sources/PINS.json`: the problem page and thread fetched on 25 August 2026, the thread snapshot of 27

August 2026 showing the correction, and the four OEIS entries. The macros of this document are written by `node tools/paper-numbers/erdos852.js`, which re-hashes every pin, re-runs the naive product, the exact-integer refutation, the battery and the verifier, re-proves the record, and refuses when a record no longer says what a sentence needs; the figure is drawn by `tools/paper-numbers/erdos852-fig.py` from the data that script derives. The report pages are `https://carlostoledo.co/reports/erdos852.html` and `https://carlostoledo.co/reports/erdos852-h.html`; repository commit `aae61990d737`.

References

[1] T. F. Bloom, Erdős Problem #852, `https://www.erdosproblems.com/852`, accessed 2026-08-25 (the page's recommended citation form); pinned as `corpus/sources/erdos852_page.html`.

[2] P. Erdős, the 1985 source paper keyed [Er85c] on the problem page [1], where the bibliographic entry is recorded; not re-verified here.

[3] Discussion thread of Erdős Problem #852, `https://www.erdosproblems.com/forum/discuss/852`: the comments of 24 April 2026 (D. Turturean; P. Chojecki) stating the conjecture and the two constants, and the correction of 26 August 2026; snapshots pinned in `corpus/sources/` (7 comments on 25 August 2026, 8 on 27 August 2026).

[4] OEIS Foundation Inc. (2026), The On-Line Encyclopedia of Integer Sequences, `https://oeis.org`: A053597, A078515, A079889 (N. J. A. Sloane, 2003, with terms by R. G. Wilson v, K. Brockhaus, D. Reble and D. Johnson, 2002–2012) and A079007 (L. Elemer, 2002, extended to 2012); entries pinned as `corpus/sources/oeis_A0*.txt`.

[5] L. Lewin, Polylogarithms and Associated Functions, North-Holland, New York, 1981.

[6] IEEE Standard for Floating-Point Arithmetic, IEEE Std 754-2019, IEEE, 2019.

[7] G. H. Hardy and E. M. Wright, An Introduction to the Theory of Numbers, 6th ed., Oxford University Press, 2008.

[8] W. Kahan, Further remarks on reducing truncation errors, Comm. ACM 8 (1965), 40.

[9] N. J. Higham, The accuracy of floating point summation, SIAM J. Sci. Comput. 14 (1993), 783–799.

[10] G. Jaeschke, On strong pseudoprimes to several bases, Math. Comp. 61 (1993), 915–926.

[11] L. de Moura and S. Ullrich, The Lean 4 theorem prover and programming language, in: Automated Deduction – CADE 28, Lecture Notes in Computer Science 12699, Springer, 2021, 625–635.

[12] The mathlib Community, The Lean mathematical library, in: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), ACM, 2020, 367–381.

[13] C. Toledo, cert-machine: independent exact certification of machine-generated mathematics, `https://github.com/carlostoledo1891/cert-machine`, concept DOI 10.5281/zenodo.22225860; the report pages are named in the reproduction section.

[14] The Ramanujan Machine results sheets, re-decided: `https://carlostoledo.co/reports/rm-audit.html`, as recorded in the repository [13] (family `families/ramanujan-audit.js`, rows `rm-zo-z5z3b-printed` and `-corrected`).

[15] The impostor catalogue: `https://carlostoledo.co/reports/impostors.html`, as recorded in the repository [13] (A271880 against 1/5).